

LAPORAN PRAKTIKUM
STRUKTUR DATA
“SINGLE LINKED LIST”
PADA BAHASA PEMROGRAMAN JAVA

disusun Oleh :

RAZIF AL FARISI

2511532028

Dosen Pengampu : Dr. Wahyudi S.T, M.T

Asisten Laboratorium : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul “**Single Linked List pada Bahasa Pemrograman Java**”. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum pada mata kuliah Struktur Data, khususnya mengenai implementasi Single Linked List pada bahasa pemrograman Java. Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep linked list, cara kerja node, pointer, serta operasi-operasi dasar yang dapat dilakukan pada Single Linked List seperti insertion, deletion, traversal, dan searching.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data yang telah memberikan ilmu dan arahan selama proses pembelajaran.

Padang, 8 Mei 2026

Penulis

Daftar Isi

KATA PENGANTAR	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
BAB II	4
PEMBAHASAN	4
2.1 Landasan Teori	4
2.2 Implementasi Program	4
2.2.1 Class Mahasiswa	4
2.2.2 Class Mahasiswa Driver	5
2.2.3 Class DaftarKata	9
2.2.4 Class DaftarKataDriver.....	11
2.3 Tugas Praktikum	15
2.3.1 Class Musik	15
2.3.2 Class Playlist (Driver Musik)	16
BAB III.....	28
PENUTUP.....	28
3.1 Kesimpulan.....	28
3.2 Saran	28
Referensi.....	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, serta mengelola data secara efisien. Pemilihan struktur data yang tepat akan sangat mempengaruhi performa program, terutama pada proses manipulasi data seperti penambahan, penghapusan, maupun pencarian data.

Berbeda dengan array yang memiliki ukuran tetap dan lokasi memori berurutan, Single Linked List bersifat dinamis sehingga ukuran data dapat bertambah maupun berkurang sesuai kebutuhan. Hal ini menjadikan Single Linked List sangat efisien dalam proses insertion dan deletion.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Single Linked List serta mampu mengimplementasikan berbagai operasi dasar pada Single Linked List menggunakan bahasa pemrograman Java.

1.2 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah sebagai berikut:

1. Memahami konsep Single Linked List dalam struktur data.
2. Memahami struktur node pada Single Linked List.
3. Mengimplementasikan operasi insertion pada Single Linked List.
4. Mengimplementasikan operasi searching pada Single Linked List.
5. Mengimplementasikan operasi deletion pada Single Linked List.
6. Memahami proses traversal pada Single Linked List.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Single Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada Single Linked List memiliki dua bagian utama, yaitu data dan pointer.

Bagian data digunakan untuk menyimpan nilai, sedangkan pointer digunakan untuk menunjuk ke node berikutnya. Node pertama disebut head, sedangkan node terakhir memiliki pointer bernilai null karena tidak menunjuk ke node lain.

Single Linked List memiliki beberapa operasi dasar, yaitu :

- Insertion, yaitu proses menambahkan node baru.
- Deletion, yaitu proses menghapus node.
- Traversal, yaitu proses menelusuri seluruh node.
- Searching, yaitu proses mencari data tertentu.

Keunggulan Single Linked List adalah fleksibilitas dalam penggunaan memori dan efisiensi pada proses penambahan maupun penghapusan data.

2.2 Implementasi Program

Berikut adalah beberapa implemenentasi *Single Linked List* :

2.2.1 Class NodeSLL_2511532028

```
package pekan5_2511532028;

public class NodeSLL_2511532028 {
    // node bagian data
    int data_2028;
    // pointer ke node berikutnya
    NodeSLL_2511532028 next_2028;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532028(int data) {
        this.data_2028 = data;
        this.next_2028 = null;
    }
}
```

Penjelasan :

Class ini merupakan class dasar yang digunakan untuk membentuk node pada Single Linked List.

```
int data_2028;
```

- Variabel ini digunakan untuk menyimpan data pada node.
- Tipe data yang digunakan adalah integer.

```
NodeSLL_2511532028 next_2028;
```

- Variabel ini merupakan pointer.
- Pointer digunakan untuk menunjuk node berikutnya.

```
public NodeSLL_2511532028(int data) {
    this.data_2028 = data;
    this.next_2028 = null;
}
}
```

- Method ini merupakan konstruktor.
- Saat object node dibuat, data yang dimasukkan akan disimpan ke variabel data_2028.
- Kemudian pointer next_2028 diisi null karena node baru belum terhubung ke node lain.

Program ini menunjukkan struktur dasar pembentukan node pada Single Linked List.

2.2.2 Class TambahSLL_2511532028

```
package pekan5_2511532028;

public class TambahSLL_2511532028 {

    public static NodeSLL_2511532028 insertAtFront_2028(NodeSLL_2511532028
head, int value) {
        NodeSLL_2511532028 new_node_2028 = new NodeSLL_2511532028(value);
        new_node_2028.next_2028 = head;
        return new_node_2028;
    }

    //fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511532028 insertAtEnd_2028(NodeSLL_2511532028 head,
int value) {
        // buat sebuah node dengan nilai
        NodeSLL_2511532028 newNode_2028 = new NodeSLL_2511532028(value);
        // jika list kosong maka node jadi head
        if (head == null) {
            return newNode_2028;
        }

        // simpan head ke variabel sementara
        NodeSLL_2511532028 last_2028 = head;
        // telusuri ke node akhir
        while(last_2028.next_2028 != null) {
            last_2028 = last_2028.next_2028;
        }

        // ubah pointer
        last_2028.next_2028 = newNode_2028;
        return head;
    }
}
```

```

    }

    static NodeSLL_2511532028 GetNode_2028(int data) {
        return new NodeSLL_2511532028(data);
    }

    static NodeSLL_2511532028 insertPos_2028(NodeSLL_2511532028 headNode, int
position, int value) {
        NodeSLL_2511532028 head_2028 = headNode;
        if(position < 1)
            System.out.print("Invalid position");
        if(position == 1) {
            NodeSLL_2511532028 new_node_2028 = new
NodeSLL_2511532028(value);
            new_node_2028.next_2028 = head_2028;
            return new_node_2028;
        } else {
            while (position-- != 0) {
                if (position == 1) {
                    NodeSLL_2511532028 newNode_2028 =
GetNode_2028(value);

                    newNode_2028.next_2028 = headNode.next_2028;
                    headNode.next_2028 = newNode_2028;
                    break;
                }
                headNode = headNode.next_2028;
            }
            if (position != 1)
                System.out.print("posisi di luar jangkauan ");
        }
        return head_2028;
    }

    public static void printList_2028(NodeSLL_2511532028 head) {
        NodeSLL_2511532028 curr_2028 = head;
        while(curr_2028.next_2028 != null) {
            System.out.print(curr_2028.data_2028+"-->");
            curr_2028 = curr_2028.next_2028;
        }
        if(curr_2028.next_2028 == null) {
            System.out.print(curr_2028.data_2028);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511532028 head_2028 = new NodeSLL_2511532028(2);
        head_2028.next_2028 = new NodeSLL_2511532028(3);
        head_2028.next_2028.next_2028 = new NodeSLL_2511532028(5);
        head_2028.next_2028.next_2028.next_2028 = new NodeSLL_2511532028(6);
        // cetak list asli
        System.out.print("senarai berantai awal : ");
        printList_2028(head_2028);
        // tambah node baru di depan
        System.out.print("tambah 1 simpul di depan : ");
        int data_2028 = 1;
        head_2028 = insertAtFront_2028(head_2028, data_2028);
        // cetak updatelist
    }

```

```

        printList_2028(head_2028);

        // tambah node baru di belakanag
        System.out.print("tambah 1 simpul di belakang : ");
        int data2_2028 = 7;
        head_2028 = insertAtEnd_2028(head_2028, data2_2028);
        // cetak update list
        printList_2028(head_2028);

        System.out.print("tambah 1 simpul ke data 4 : ");
        int data3_2028 = 4;
        int pos_2028 = 4;
        head_2028 = insertPos_2028(head_2028, pos_2028, data3_2028);
        // cetak update list
        printList_2028(head_2028);
    }
}

```

Penjelasan :

Class ini digunakan untuk menambahkan node pada Single Linked List.

```

public static NodeSLL_2511532028 insertAtFront_2028(NodeSLL_2511532028 head, int
value) {
    NodeSLL_2511532028 new_node_2028 = new NodeSLL_2511532028(value);
    new_node_2028.next_2028 = head;
    return new_node_2028;
}

```

- Method ini digunakan untuk menambahkan node baru di bagian depan linked list.
- Pointer node baru di arahkan ke head lama.
- Node baru dikembalikan sebagai head yang baru.

```

public static NodeSLL_2511532028 insertAtEnd_2028(NodeSLL_2511532028 head, int
value) {
    // buat sebuah node dengan nilai
    NodeSLL_2511532028 newNode_2028 = new NodeSLL_2511532028(value);
    // jika list kosong maka node jadi head
    if (head == null) {
        return newNode_2028;
    }

    // simpan head ke variabel sementara
    NodeSLL_2511532028 last_2028 = head;
    // telusuri ke node akhir
    while(last_2028.next_2028 != null) {
        last_2028 = last_2028.next_2028;
    }

    // ubah pointer
    last_2028.next_2028 = newNode_2028;
    return head;
}

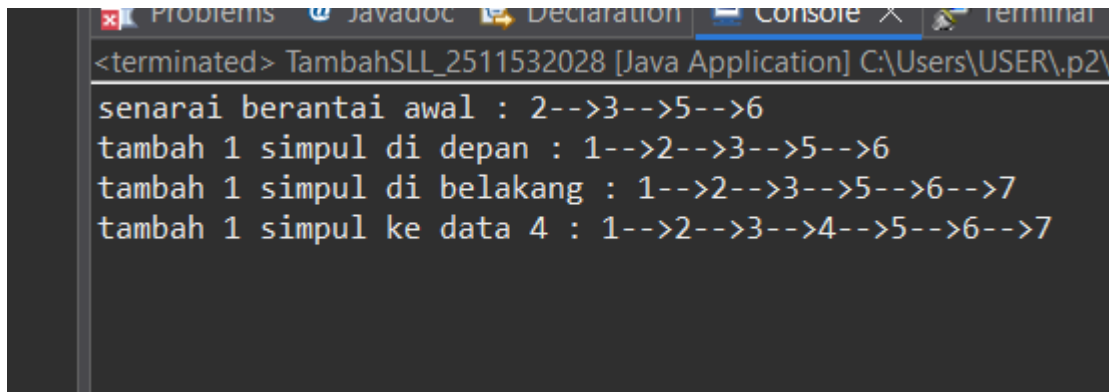
```


- Method ini digunakan untuk menambahkan node di bagian akhir linked list.
- Program terlebih dahulu di cek apakah list kosong, jika (head = null) berarti belum ada node. Maka node baru langsung dijadikan head.
- Terdapat variable bantu untuk traversal(last_2028).
- Perulangan untuk menelusuri node satu per satu sampai terakhir.
- Pointer node terakhir dihubungkan ke node baru.

```
static NodeSLL_2511532028 insertPos_2028(NodeSLL_2511532028 headNode, int
position, int value) {
    NodeSLL_2511532028 head_2028 = headNode;
    if(position < 1)
        System.out.print("Invalid position");
    if(position == 1) {
        NodeSLL_2511532028 new_node_2028 = new
NodeSLL_2511532028(value);
        new_node_2028.next_2028 = head_2028;
        return new_node_2028;
    } else {
        while (position-- != 0) {
            if (position == 1) {
                NodeSLL_2511532028 newNode_2028 =
GetNode_2028(value);
                newNode_2028.next_2028 = headNode.next_2028;
                headNode.next_2028 = newNode_2028;
                break;
            }
            headNode = headNode.next_2028;
        }
        if (position != 1)
            System.out.print("posisi di luar jangkauan ");
    }
    return head_2028;
}
```

- Method ini untuk menambahkan node pada posisi tertentu.
- Program apakah posisi < 1, jika true posisi tidak valid.
- Jika posisi = 1, node yang ditambahkan dijadikan head.
- Perulangan menelusuri node sampai posisi yang diminta.

Gambar 2.2.2



```
<terminated> TambahSLL_2511532028 [Java Application] C:\Users\USER\p2\
senarai berantai awal : 2-->3-->5-->6
tambah 1 simpul di depan : 1-->2-->3-->5-->6
tambah 1 simpul di belakang : 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4 : 1-->2-->3-->4-->5-->6-->7
```

2.2.3 Class PencarianSLL_2511532028

```
package pekan5_2511532028;

public class PencarianSLL_2511532028 {
    static boolean searchKey_2028(NodeSLL_2511532028 head, int key) {
        NodeSLL_2511532028 curr_2028 = head;
        while (curr_2028 != null) {
            if (curr_2028.data_2028 == key)
                return true;
            curr_2028 = curr_2028.next_2028;
        }
        return false;
    }

    public static void traversal(NodeSLL_2511532028 head) {
        // mulai dari head
        NodeSLL_2511532028 curr_2028 = head;
        // telusuri sampai pointer null
        while (curr_2028 != null) {
            System.out.print(" " + curr_2028.data_2028);
            curr_2028 = curr_2028.next_2028;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511532028 head_2028 = new NodeSLL_2511532028(14);
        head_2028.next_2028 = new NodeSLL_2511532028(21);
        head_2028.next_2028.next_2028 = new NodeSLL_2511532028(13);
        head_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(30);
        head_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(10);
        System.out.print("Penelusuran SLL : ");
        traversal(head_2028);
        // data yang akan di cari
        int key_2028 = 30;
        System.out.print("cari data " + key_2028 + " = ");
        if (searchKey_2028(head_2028, key_2028))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada ");
    }
}
```

```
}
```

Penjelasan :

```
static boolean searchKey_2028(NodeSLL_2511532028 head, int key) {  
    NodeSLL_2511532028 curr_2028 = head;  
    while (curr_2028 != null) {  
        if (curr_2028.data_2028 == key)  
            return true;  
        curr_2028 = curr_2028.next_2028;  
    }  
    return false;  
}
```

- Program menelusuri node mulai dari head.
- Jika data yang di cari sesuai data yang ada di list, return true. Jika tidak ditemukan hingga node terakhir, berarti curr = null(return false).

```
NodeSLL_2511532028 head_2028 = new NodeSLL_2511532028(14);  
head_2028.next_2028 = new NodeSLL_2511532028(21);  
head_2028.next_2028.next_2028 = new NodeSLL_2511532028(13);  
head_2028.next_2028.next_2028.next_2028 = new  
NodeSLL_2511532028(30);  
head_2028.next_2028.next_2028.next_2028.next_2028 = new  
NodeSLL_2511532028(10);
```

- Membuat node pertama dan menjadikannya head(14).
- Setelah itu pointer node 14 diarahkan ke node 21, lanjut di arahkan ke node 13, ke node 30, dan terakhir ke node 10.

```
System.out.print("Penelusuran SLL : ");  
traversal(head_2028);
```

Method ini akan:

- Mulai dari head.
- Menampilkan data.
- Berpindah ke node berikutnya.
- Berhenti saat pointer null.

```
int key_2028 = 30;  
System.out.print("cari data " + key_2028 + " = ");  
if (searchKey_2028(head_2028, key_2028))  
    System.out.println("ketemu");  
else  
    System.out.println("tidak ada");  
}
```

- Key adalah data yang mau di cari (30). Jika data 30 di temukan, program return true dan di kirim ke if(print “ketemu”). Jika tidak di temukan, makan program return false ke if (print “tidak ada”).

Output :

Gambar 2.2.3

```
<terminated> PencarianSLL_2511532028 [Java Application]
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

2.2.4 Class HapusSLL_2511532028

```
package pekan5_2511532028;

public class HapusSLL_2511532028 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511532028 deleteHead_2028(NodeSLL_2511532028 head) {
        // jika SLL kosong
        if (head == null)
            return null;
        // pindahkan head ke node berikutnya
        head = head.next_2028;
        // return head ke node berikutnya
        return head;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532028 removeLastNode(NodeSLL_2511532028 head) {
        // jika list kosong, return null
        if (head == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head.next_2028 == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511532028 secondLast_2028 = head;
        while (secondLast_2028.next_2028.next_2028 != null) {
            secondLast_2028 = secondLast_2028.next_2028;
        }
        // hapus node terakhir
        secondLast_2028.next_2028 = null;
        return head;
    }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511532028 deleteNode_2028(NodeSLL_2511532028 head,
int position) {
        NodeSLL_2511532028 temp_2028 = head;
        NodeSLL_2511532028 prev_2028 = null;
        // jika linked list null
        if (temp_2028 == null)
            return head;
        // kasus 1 : head di hapus
        if (position == 1) {
            head = temp_2028.next_2028;
            return head;
        }
    }
}
```

```

// kasus 2 : menghapus node di tengah
// telusuri ke node yang di hapus

for(int i = 1; temp_2028 != null && i < position; i++) {
    prev_2028 = temp_2028;
    temp_2028 = temp_2028.next_2028;
}
// jika di temukan, hapus node
if (temp_2028 != null) {
    prev_2028.next_2028 = temp_2028.next_2028;
} else {
    System.out.println("Data tidak ada ");
}
return head;
}
// fungsi mencetak SLL
public static void printList_2028(NodeSLL_2511532028 head) {
    NodeSLL_2511532028 curr_2028 = head;
    while (curr_2028.next_2028 != null) {
        System.out.print(curr_2028.data_2028 + " --> ");
        curr_2028 = curr_2028.next_2028;
    }
    if (curr_2028.next_2028 == null) {
        System.out.print(curr_2028.data_2028);
    }
    System.out.println();
}

// kelas main
public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5
    NodeSLL_2511532028 head_2028 = new NodeSLL_2511532028(1);
    head_2028.next_2028 = new NodeSLL_2511532028(2);
    head_2028.next_2028.next_2028 = new NodeSLL_2511532028(3);
    head_2028.next_2028.next_2028.next_2028 = new NodeSLL_2511532028(4);
    head_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(5);
    head_2028.next_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(6);
    // cetak list awal
    System.out.println("List awal :");
    printList_2028(head_2028);
    // hapus head
    head_2028 = deleteHead_2028(head_2028);
    System.out.println("List setelah head di hapus : ");
    printList_2028(head_2028);
    // hapus node terakhir
    head_2028 = removeLastNode(head_2028);
    System.out.println("List setelah simpul terakhir di hapus : ");
    printList_2028(head_2028);
    // hapus node pada posisi 2
    int position_2028 = 2;
    head_2028 = deleteNode_2028(head_2028, position_2028);
    // print list setelah di hapus
    System.out.println("List setelah posisi 2 dihapus : ");
    printList_2028(head_2028);
}
}

```

Penjelasan :

```
public static NodeSLL_2511532028 deleteHead_2028(NodeSLL_2511532028 head) {  
    // jika SLL kosong  
    if (head == null)  
        return null;  
    // pindahkan head ke node berikutnya  
    head = head.next_2028;  
    // return head ke node berikutnya  
    return head;  
}
```

- Program awalnya mengecek apakah list kosong.
- Jika false, head dipindahkan dan mengembalikan nilai head

```
public static NodeSLL_2511532028 removeLastNode(NodeSLL_2511532028 head) {  
    // jika list kosong, return null  
    if (head == null) {  
        return null;  
    }  
    // jika list satu node, hapus node dan return null  
    if (head.next_2028 == null) {  
        return null;  
    }  
    // temukan node terakhir ke dua  
    NodeSLL_2511532028 secondLast_2028 = head;  
    while (secondLast_2028.next_2028.next_2028 != null) {  
        secondLast_2028 = secondLast_2028.next_2028;  
    }  
    // hapus node terakhir  
    secondLast_2028.next_2028 = null;  
    return head;  
}
```

- Jika head kosong, return null.
- Jika hanya ada 1 node, return null juga.
- Setelah itu mencari node kedua terakhir dengan perulangan while, lalu putuskan pointer(node terakhir di hapus).

```
public static NodeSLL_2511532028 deleteNode_2028(NodeSLL_2511532028 head, int position) {  
    NodeSLL_2511532028 temp_2028 = head;  
    NodeSLL_2511532028 prev_2028 = null;  
    // jika linked list null  
    if (temp_2028 == null)  
        return head;  
    // kasus 1 : head di hapus  
    if (position == 1) {  
        head = temp_2028.next_2028;  
        return head;  
    }  
    // kasus 2 : menghapus node di tengah  
    // telusuri ke node yang di hapus  
    for(int i = 1; temp_2028 != null && i < position; i++) {  
        prev_2028 = temp_2028;  
        temp_2028 = temp_2028.next_2028;  
    }  
}
```

```

// jika di temukan, hapus node
if (temp_2028 != null) {
    prev_2028.next_2028 = temp_2028.next_2028;
} else {
    System.out.println("Data tidak ada ");
}
return head;
}

```

- Method ini menghapus node pada posisi tertentu.
- Jika posisi yang mau di hapus = 1, head di hapus.
- Kalau posisi tengah yang di hapus, pointer prev menyimpan node sebelumnya, dan temp menyimpan node target.
- Pointer langsung melompati node target.

```

public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5
    NodeSLL_2511532028 head_2028 = new NodeSLL_2511532028(1);
    head_2028.next_2028 = new NodeSLL_2511532028(2);
    head_2028.next_2028.next_2028 = new NodeSLL_2511532028(3);
    head_2028.next_2028.next_2028.next_2028 = new NodeSLL_2511532028(4);
    head_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(5);
    head_2028.next_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeSLL_2511532028(6);
}

```

- Membuat objek dengan node pertama bernilai 1, setelah itu head.next diberi nilai 2 hingga 6.

```

head_2028 = deleteHead_2028(head_2028);
System.out.println("List setelah head di hapus : ");
printList_2028(head_2028);

```

- Method ini menghapus node awal / head lalu menampilkan list.

```

head_2028 = removeLastNode(head_2028);
System.out.println("List setelah simpul terakhir di hapus : ");
printList_2028(head_2028);

```

- Method ini menghapus node terakhir menggunakan variable pembantu (prev dan temp).

```

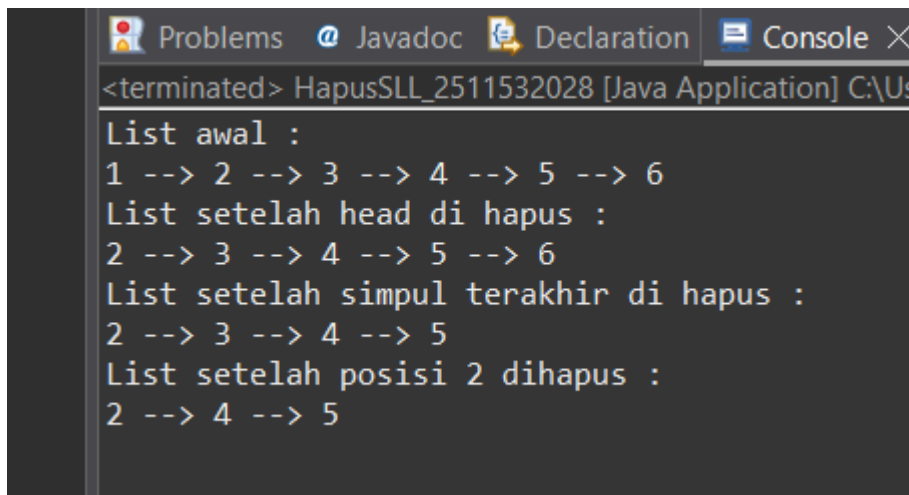
int position_2028 = 2;
head_2028 = deleteNode_2028(head_2028, position_2028);
// print list setelah di hapus
System.out.println("List setelah posisi 2 dihapus : ");
printList_2028(head_2028);
}

```

- Menghapus node sesuai nilai pada variable posisi (2) / posisi tertentu.

Output :

Gambar 2.2.4



```
<terminated> HapusSLL_2511532028 [Java Application] C:\Us
List awal :
1 --> 2 --> 3 --> 4 --> 5 --> 6
List setelah head di hapus :
2 --> 3 --> 4 --> 5 --> 6
List setelah simpul terakhir di hapus :
2 --> 3 --> 4 --> 5
List setelah posisi 2 dihapus :
2 --> 4 --> 5
```

2.3 Tugas Praktikum

2.3.1 Class Pasien_2511532028

```
package pekan5_2511532028;

import java.util.Scanner;

public class Pasien_2511532028 {
    Scanner s = new Scanner(System.in);
    String pasien_2028;
    String keluhan_2028;
    int nomor_antri_2028;
    static int counter_2028;
    Pasien_2511532028 next_2028;

    public Pasien_2511532028(String pasien, String keluhan) {
        this.nomor_antri_2028 = ++counter_2028;
        this.pasien_2028 = pasien;
        this.keluhan_2028 = keluhan;
        this.next_2028 = null;
    }
}
```

Penjelasan :

Class ini digunakan sebagai node pada Single Linked List. Setiap node menyimpan data pasien.

Setiap node memiliki beberapa bagian :

- Nama pasien
- Keluhan pasien
- Nomor antrian
- Pointer ke node berikutnya


```
int nomor_antri_2028;
static int counter_2028;
```

- Digunakan untuk menyimpan nomor urut pasien.
- Variable static digunakan untuk menghitung jumlah node yang pernah di buat karena static, nilainya akan di pakai bersama oleh seluruh object.

```
public Pasien_2511532028(String pasien, String keluhan) {
    this.nomor_antri_2028 = ++counter_2028;
    this.pasien_2028 = pasien;
    this.keluhan_2028 = keluhan;
    this.next_2028 = null;
}
```

- Konstruktor akan di jalankan saat object pasien di buat
- Nomor antrian otomatis karena nilai counter di naikan.
- Nama input user di simpan.
- Dan next bernilai null karena node baru belum terhubung.

2.3.2 Class RumahSakit_2511532028

```
package pekan5_2511532028;

import java.util.Scanner;

public class RumahSakit_2511532028 {

    public static void MenuRS_2028() {
        System.out.println("\n====Menu Antrian Rumah Sakit====");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Pasien");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
    }

    static boolean cariPasien_2028(Pasien_2511532028 pasien, Scanner s) {
        Pasien_2511532028 bantu_2028 = pasien;
        if (pasien == null) {
            return false;
        }
        System.out.println("====Pencarian Nama Pasien====");
        System.out.print("Nama Pasien : ");
        String caripasien_2028 = s.nextLine();
        System.out.println("=====\n");
        while (bantu_2028 != null) {
            if (caripasien_2028.equalsIgnoreCase(bantu_2028.pasien_2028))
            {
                System.out.println("====Data
                Pasien====");
                System.out.println("Nama Pasien : " +
                bantu_2028.pasien_2028);
            }
        }
    }
}
```

```

        System.out.println("Keluhan Pasien      : " +
bantu_2028.keluhan_2028 );
        return true;
    }
    bantu_2028 = bantu_2028.next_2028;
}
return false;
}

static void statusAntri_2028(Pasien_2511532028 pasien) {
    Pasien_2511532028 status_2028 = pasien;
    int banyakpasien_2028 = 0;
    System.out.println("====Status Antrian====");
    if (pasien == null) {
        System.out.println("===={Pasien Tidak Ada}====");
        return ;
    }
    while (status_2028 != null) {
        ++banyakpasien_2028;
        status_2028 = status_2028.next_2028;
    }
    System.out.print("Banyak Pasien : ");
    System.out.println(banyakpasien_2028);
}

public static void printAntri_2028(Pasien_2511532028 pasien) {
    Pasien_2511532028 antri_2028 = pasien;
    if (pasien == null) {
        System.out.println("Antrian kosong");
        return;
    }
    System.out.println("====Antrian Rumah Sakit====");
    while(antri_2028.next_2028 != null) {
        System.out.print(antri_2028.nomor_antri_2028 + ". " +
antri_2028.pasien_2028 +
        " [" +antri_2028.keluhan_2028 + "]" + " --> ");
        antri_2028 = antri_2028.next_2028;
    }
    if(antri_2028.next_2028 == null) {
        System.out.print(antri_2028.nomor_antri_2028 + ". "
+ antri_2028.pasien_2028 + " [" +antri_2028.keluhan_2028 + "]"");
    }
    System.out.println();
}

public static Pasien_2511532028 MasukanPasien_2028(Pasien_2511532028
pasien, Scanner s) {
    //
    String nama_2028;
    String keluhan_2028;
    System.out.println("====Data Pasien====");
    System.out.print("Nama Pasien      : ");
    nama_2028 = s.nextLine();
    nama_2028 = nama_2028.substring(0,1).toUpperCase() +
nama_2028.substring(1);
    System.out.print("Keluhan Pasien : ");

```

```

        keluhan_2028 = s.nextLine();
        System.out.println();
        Pasien_2511532028 antribaru_2028 = new Pasien_2511532028(nama_2028,
keluhan_2028);
        if (pasien == null) {
            antribaru_2028.nomor_antri_2028 = 1;
            return antribaru_2028;
        }
        Pasien_2511532028 lastantri = pasien;
        while (lastantri.next_2028 != null) {
            lastantri = lastantri.next_2028;
        }
        antribaru_2028.nomor_antri_2028 =
            lastantri.nomor_antri_2028 + 1;
        lastantri.next_2028 = antribaru_2028;
        return pasien;
    }

    public static Pasien_2511532028 panggilPasien_2028(Pasien_2511532028
pasien) {
        if (pasien == null) {
            System.out.println("Antrian Kosong ");
            return null;
        }
        // head = head.next
        System.out.println("====Data Pasien====");
        System.out.println("Nama Pasien : " + pasien.pasien_2028);
        System.out.println("Keluhan Pasien : " + pasien.keluhan_2028 );
        System.out.println("Status Pasien : Sudah mendapat pelayanan
atau pengobatan");
        pasien = pasien.next_2028;
        Pasien_2511532028 bantu = pasien;
        while (bantu != null) {
            bantu.nomor_antri_2028--;
            bantu = bantu.next_2028;
        }
        return pasien;
    }

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        Pasien_2511532028 pasien_2028 = null;
        int pilihan_2028;
        do {
            MenuRS_2028();
            System.out.print("Pilih menu : ");
            pilihan_2028 = s.nextInt();
            System.out.println();
            s.nextLine();

            switch (pilihan_2028) {
                case 1:
                    pasien_2028 = MasukanPasien_2028(pasien_2028,s);
                    break;
                case 2:

```

```

        pasien_2028 = panggilPasien_2028(pasien_2028);
        break;
    case 3:
        printAntri_2028(pasien_2028);
        break;
    case 4:
        if (cariPasien_2028(pasien_2028,s))

        System.out.println("====={ketemu}=====");
        else
            System.out.println("=={pasien tidak ada
atau antrian kosong}===");
        break;
    case 5:
        statusAntri_2028(pasien_2028);
        break;
    case 6:
        System.out.println("==Anda Keluar dari Menu
Antrian===");
        break;
    }

    } while (pilihan_2028 != 6);
}

```

Penjelasan :

```

public static void MenuRS_2028() {
    System.out.println("\n====Menu Antrian Rumah Sakit====");
    System.out.println("1. Daftarkan Pasien");
    System.out.println("2. Panggil Pasien");
    System.out.println("3. Tampilkan Pasien");
    System.out.println("4. Cari Pasien");
    System.out.println("5. Cek Status Antrian");
    System.out.println("6. Keluar");
}

```

- Method ini untuk menampilkan menu yang ada di rumah sakit.

```

static boolean cariPasien_2028(Pasien_2511532028 pasien, Scanner s) {
    Pasien_2511532028 bantu_2028 = pasien;
    if (pasien == null) {
        return false;
    }
    System.out.println("====Pencarian Nama Pasien====");
    System.out.print("Nama Pasien : ");
    String caripasien_2028 = s.nextLine();
    System.out.println("=====\n");
    while (bantu_2028 != null) {
        if (caripasien_2028.equalsIgnoreCase(bantu_2028.pasien_2028))
        {
            System.out.println("====Data
Pasien====");
            System.out.println("Nama Pasien : " +
bantu_2028.pasien_2028);
            System.out.println("Keluhan Pasien : " +
bantu_2028.keluhan_2028 );
            return true;
        }
    }
}

```

```

    }
    bantu_2028 = bantu_2028.next_2028;
}
return false;
}

```

- Method ini untuk mengecek apakah pasien yang di cari ada atau tidak.
- Awalnya dimulai dari head, saat pasien null, return false(lansung menyelesaikan method).
- Kalau pasien tidak null, program akan menelusuri seluruh node dengan traversal.
- Lalu bandingkan nama yang dicari dengan nama yang ada pada node tanpa membedakan huruf besar kecil.
- Jika ketemu, method mengembalikan nilai true.
- Jika tidak, return nilai false;

```

static void statusAntri_2028(Pasien_2511532028 pasien) {
    Pasien_2511532028 status_2028 = pasien;
    int banyakpasien_2028 = 0;
    System.out.println("=====Status Antrian=====");
    if (pasien == null) {
        System.out.println("===={Pasien Tidak Ada}====");
        return ;
    }
    while (status_2028 != null) {
        ++banyakpasien_2028;
        status_2028 = status_2028.next_2028;
    }
    System.out.print("Banyak Pasien : ");
    System.out.println(banyakpasien_2028);
}

```

- Banyak pasien dimuali dari nol.
- Jika list atau status_2028 tidak null, program menghitung node(++banyakpasien_2028).
- Program membaca node selanjutnya.
- Lalu tampilkan banyak pasien.

```

public static void printAntri_2028(Pasien_2511532028 pasien) {
    Pasien_2511532028 antri_2028 = pasien;
    if (pasien == null) {
        System.out.println("Antrian kosong");
        return;
    }
    System.out.println("=====Antrian Rumah Sakit=====");
    while(antri_2028.next_2028 != null) {
        System.out.print(antri_2028.nomor_antri_2028 + ". " +
antri_2028.pasien_2028 +
        " [" +antri_2028.keluhan_2028 + "]" + " --> ");
        antri_2028 = antri_2028.next_2028;
    }
    if(antri_2028.next_2028 == null) {

```

```

        System.out.print(antri_2028.nomor_antri_2028 + ". "
+ antri_2028.pasien_2028 + " [" + antri_2028.keluhan_2028 + "]);
    }
    System.out.println();
}

```

- Method menampilkan list yang sudah di buat.
- Jika pasien null, berarti list masih kosong.
- Variable antri sebagai pointer traversal.
- Program melakukan perulangan hingga node terakhir bernilai null, lalu menampilkan nomor, nama , dan keluhan dari pasien.
- Lalu pointer berpindah ke node selanjutnya.

```

public static Pasien_2511532028 MasukanPasien_2028(Pasien_2511532028
pasien, Scanner s) {
    //
    String nama_2028;
    String keluhan_2028;
    System.out.println("=====Data Pasien=====");
    System.out.print("Nama Pasien    : ");
    nama_2028 = s.nextLine();
    nama_2028 = nama_2028.substring(0,1).toUpperCase() +
nama_2028.substring(1);
    System.out.print("Keluhan Pasien : ");
    keluhan_2028 = s.nextLine();
    System.out.println();
    Pasien_2511532028 antribaru_2028 = new Pasien_2511532028(nama_2028,
keluhan_2028);
    if (pasien == null) {
        antribaru_2028.nomor_antri_2028 = 1;
        return antribaru_2028;
    }
    Pasien_2511532028 lastantri = pasien;
    while (lastantri.next_2028 != null) {
        lastantri = lastantri.next_2028;
    }
    antribaru_2028.nomor_antri_2028 =
        lastantri.nomor_antri_2028 + 1;
    lastantri.next_2028 = antribaru_2028;
    return pasien;
}

```

- Method ini merupakan enqueue / menambahkan data pasien.
- Variable nama dan keluhan menggunakan inputan dari scanner. Untuk nama yang diinput, di dalam list huruf awal dari nama otomatis kapital(uppercase).
- Membuat node baru.
- Jika list null / pasien null, pasien yang pertama di input akan mendapat nomor antrian pertama. Lalu node tersebut menjadi head.
- Membuat pointer traversal yang bergerak sampai node terakhir.

- Nomor antri dari antribaru = nomor antri dari last antri++(increasement).
- Lalu node disambungkan.

```

public static Pasien_2511532028 panggilPasien_2028(Pasien_2511532028
pasien) {
    if (pasien == null) {
        System.out.println("Antrian Kosong ");
        return null;
    }
    // head = head.next
    System.out.println("====Data Pasien====");
    System.out.println("Nama Pasien : " + pasien.pasien_2028);
    System.out.println("Keluhan Pasien : " + pasien.keluhan_2028 );
    System.out.println("Status Pasien : Sudah mendapat pelayanan
atau pengobatan");
    pasien = pasien.next_2028;
    Pasien_2511532028 bantu = pasien;
    while (bantu != null) {
        bantu.nomor_antri_2028--;
        bantu = bantu.next_2028;
    }
    return pasien;
}

```

- Jika pasien null, makan antrian kosong dan return null.
- Head atau pointer awal di geser ke node selanjutnya.
- Lalu traversal seluruh node, nomor antrian dikurangi.
- Lalu return pasien.

```

public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    Pasien_2511532028 pasien_2028 = null;
    int pilihan_2028;
    do {
        MenuRS_2028();
        System.out.print("Pilih menu : ");
        pilihan_2028 = s.nextInt();
        System.out.println();
        s.nextLine();

        switch (pilihan_2028) {
            case 1:
                pasien_2028 = MasukanPasien_2028(pasien_2028,s);
                break;
            case 2:
                pasien_2028 = panggilPasien_2028(pasien_2028);
                break;
            case 3:
                printAntri_2028(pasien_2028);
                break;
            case 4:
                if (cariPasien_2028(pasien_2028,s))

        System.out.println("====={ketemu}====");
        else
    }
}

```

```

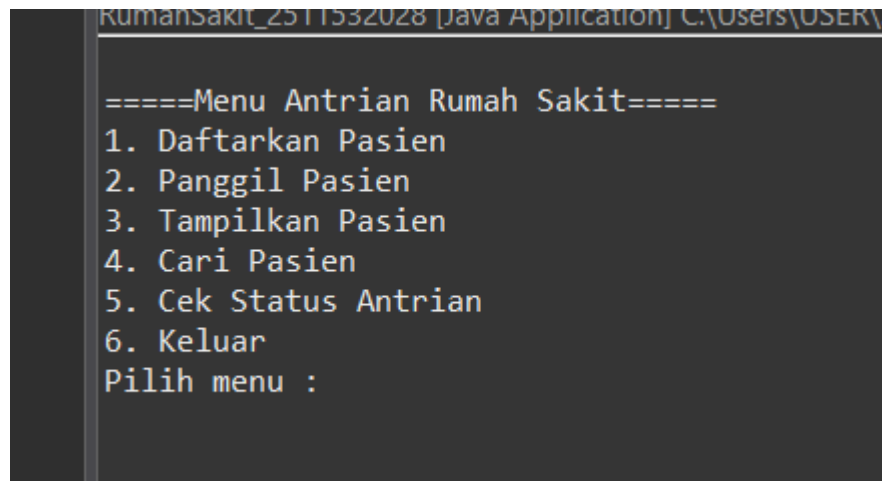
                                System.out.println("=={pasien tidak ada
atau antrian kosong}==");
                                break;
                                case 5:
                                    statusAntri_2028(pasien_2028);
                                    break;
                                case 6:
                                    System.out.println("==Anda Keluar dari Menu
Antrian==");
                                    break;
                                }
                                } while (pilihan_2028 != 6);
                                }
}

```

- Method awal inialisasi head awal kosong
- variable pilihan untuk memilih menu yang ingin di jalankan.
- **Saat pilihan bernilai 1**, tambah node. Kalau list masih kosong, node menjadi head.
- **Saat pilihan bernilai 2**, hapus node/pasien.
- **Pilihan = 3**, untuk menampilkan node-node yang ada di dalam list.
- **Saat pilihan = 4**, cari pasien berdasarkan nama tanpa melihat huruf besar kecil, jika nama pasien yang dicari ada di dalam list, print data pasien. Jika tidak, print pasien tidak di temukan.
- **Pilihan 5**, untuk melihat status antrian, berapa banyak pasien yang mengantri. Jika list kosong, print antrian kosong atau pasien belum ada.
- **Pilihan 6**, untuk keluar dari menu.

Gambar 2.3.2

Tampilan dari menu Rumah Sakit :



```

RumahSakit_2511532028 [Java Application] C:\Users\USER\
====Menu Antrian Rumah Sakit====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu :

```


Menambahkan pasien :

```
RumahSakit_2511532028 [Java Application] C:\Users\USER\p2\pool\pl

=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 1

=====Data Pasien=====
Nama Pasien      : budi
Keluhan Pasien   : demam
|
```

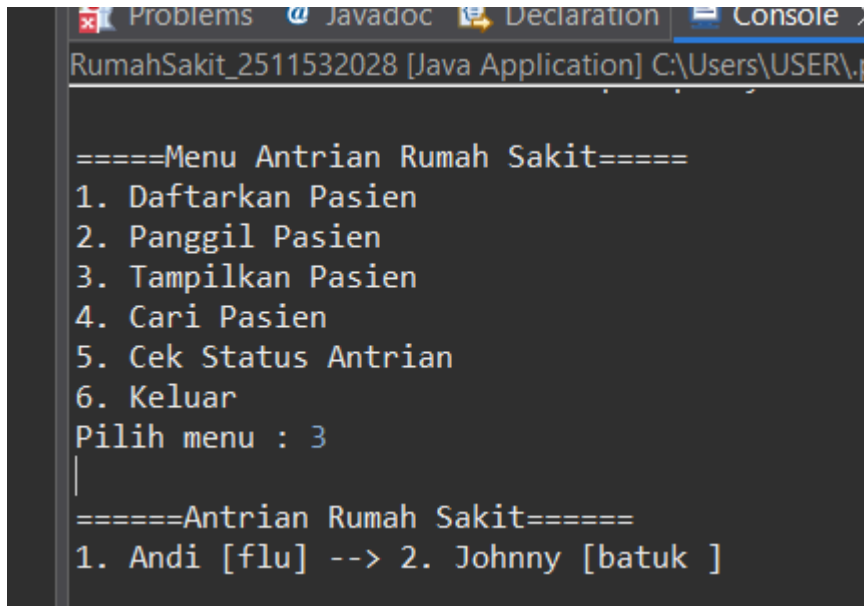
Menghapus Pasien / pasien sudah diberikan pengobatan :

```
RumahSakit_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse

=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 2

=====Data Pasien=====
Nama Pasien      : Budi
Keluhan Pasien   : demam
Status Pasien    : Sudah mendapat pelayanan atau pengobatan
```

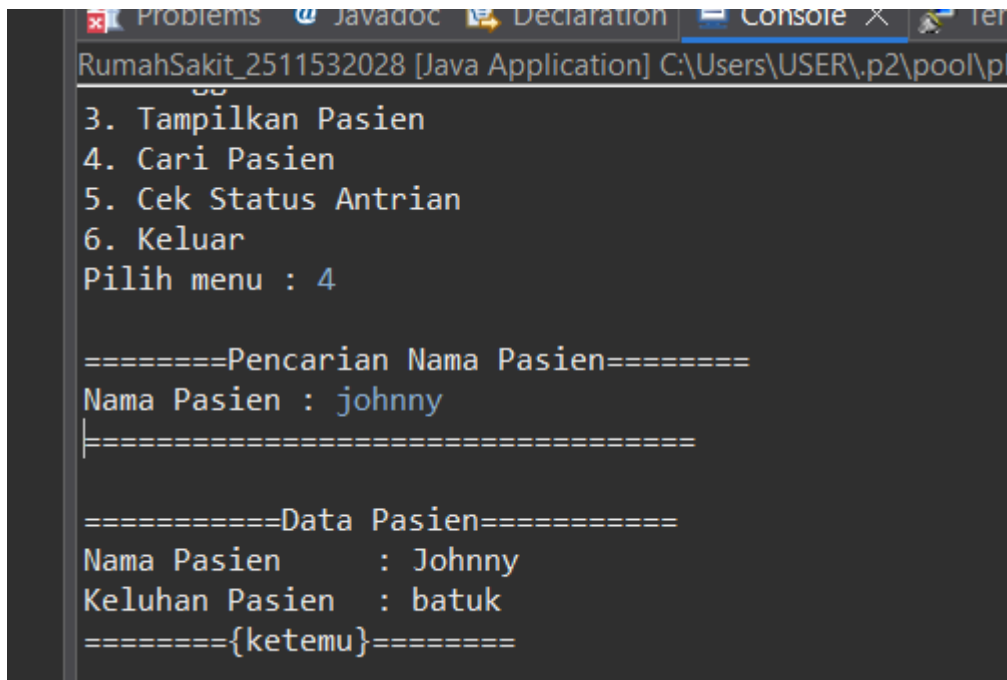
Menampilkan antrian di Rumah Sakit (sesudah hapus pasien) :



```
Problems Javadoc Declaration Console
RumahSakit_2511532028 [Java Application] C:\Users\USER\p2\pool\pl

=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 3
|
=====Antrian Rumah Sakit=====
1. Andi [flu] --> 2. Johnny [batuk ]
```

Mencari nama pasien :



```
Problems Javadoc Declaration Console X Ter
RumahSakit_2511532028 [Java Application] C:\Users\USER\p2\pool\pl

3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 4

=====Pencarian Nama Pasien=====
Nama Pasien : johnny
|=====

=====Data Pasien=====
Nama Pasien      : Johnny
Keluhan Pasien   : batuk
===== {ketemu} =====
```

Melihat status antrian :

```
=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 5
|
=====Status Antrian=====
Banyak Pasien : 2
```

Saat list masih kosong :

```
6. Keluar
Pilih menu : 2

Antrian Kosong

=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
```

```
=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 3

Antrian kosong
```

```
6. Keluar
Pilih menu : 4

==={pasien tidak ada atau antrian kosong}===

=====Menu Antrian Rumah Sakit=====
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Pasien
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu : 5

=====Status Antrian=====
===={Pasien Tidak Ada}=====
```

Program ini berhasil mengimplementasikan **Queue menggunakan Single Linked List**, di mana pasien pertama yang masuk akan menjadi pasien pertama yang dipanggil sesuai prinsip **FIFO (First In First Out)**. Selain enqueue dan dequeue, program juga dapat melakukan pencarian data, traversal, dan pengecekan jumlah node dalam antrian.

BAB III

PENUTUP

3.1 Kesimpulan

Single Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung menggunakan pointer. Setiap node memiliki data dan pointer yang menunjuk ke node berikutnya.

Melalui praktikum ini, dapat dipahami bagaimana proses pembentukan node, penambahan data, pencarian data, penghapusan data, serta traversal pada Single Linked List menggunakan bahasa pemrograman Java.

Dengan memahami konsep Single Linked List, mahasiswa dapat lebih mudah mengimplementasikan struktur data dinamis dalam berbagai permasalahan pemrograman.

3.2 Saran

Dalam mempelajari Single Linked List, disarankan untuk memahami konsep dasar node, pointer, dan head terlebih dahulu sebelum mengimplementasikan program.

Selain itu, latihan yang beragam seperti insertion, deletion, searching, dan traversal akan membantu meningkatkan pemahaman terhadap struktur data linked list.

Referensi

Data Structures and Algorithms in Java

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.

Oracle. *Java Collections Framework*.